📑 Agenda for Week1 day 02 - Operators and Control Statements

Operators

Conditional Statements

Break

Looping Statements

Break out

Strings

Operators

Operators are special symbols used to perform operations on values (operands).

ex: |

```
let a = 10;
let b = 5;

console.log(a + b);
```

+ → Operator

a, b → Operands

a) Arithmetic Operators

Used for mathematical calculations.

Operator	Description	Example
+	Addition	10 + 5 = 15

- Subtraction 10 - 5 = 5

* Multiplication 10 * 5 = 50

/ Division 10 / 5 = 2

% Modulus 10 % 3 = 1

Example*:*

Division Operator*:*

10 / 5 = 2 //Returns Quotient

Modulus Operator*:*

10 % 3 = 1 → Returns Remainder

b) Assignment Operators

Used to assign or update values in variables.

Operator	Example	Meaning
=	a = 10	Assign value
+=	a += 5	a = a + 5
-=	a -= 5	a = a - 5
*=	a *= 5	a = a * 5
/=	a /= 5	a = a / 5
%=	a %= 5	a = a % 5

Example*:*

```
let count = 10;
count += 5;  //count=count+5 => 10+5=>15
```

```
console.log(count);
```

Output:

15

c) Comparison Operators

Used to compare values and returns Boolean value:

true

false

Operator	Description
>	Greater than
<	Lesser than
>=	Greater than or equal
<=	Lesser than or equal
===	Strict Equality
==	Loose Equality

Strict Equality (===):

Checks both the value and the Data Type

Example:

```
console.log(10 === "10");  //number 10 is not equal to string value "10"
```

Output:

false

Loose Equality (==):

Checks only value.

JavaScript performs type coercion automatically

coercion means JavaScript converts string "1" into number 1 before comparison

Example*:*

```
console.log(1 == "1");
```

Output*:*

true

d) Logical Operators

Used to combine multiple conditions and returns boolean value

true

false

Example*:*

```
let a = true;
let b = false;
```

AND Operator (&&)*:*

Returns true only when both conditions are true.

```
console.log(a && b);
```

Output*:*

false

Example*:*

let age = 25;

console.log(age > 18 && age < 60);

Output*:*

true

OR Operator (||)*:*

Returns true if at least one condition is true.

console.log(a || b);

Output*:*

true

NOT Operator (!)*:*

Reverses the boolean value.

console.log(!true);

Output*:*

false

e) Unary Operators

Unary operators work on a single operand (single variable).

Increment Operator (++)*:*

Used to increase value by 1.

Pre-Increment (++a)

Increment first

Then use the value

Example*:*

```
let a = 5;
console.log(++a);
```

Output*:*

6

Post-Increment (a++)

Use the value first

Then increment

Example*:*

```
let a = 5;

console.log(a++);
console.log(a);
```

Output*:*

5

6

Decrement Operator (--)*:*

Used to decrease value by 1.

Pre-Decrement (--a)

Decrement first

Then use the value

let b = 5;

console.log(--b);

Output*:*

4

Post-Decrement (b--)

Use the value first

Then decrement

let b = 5;

console.log(b--);

console.log(b);

Output*:*

5

4

Control Statements in JavaScript*:*

Control statements are used to control the flow of execution of a program

By default, JavaScript executes code from top to bottom (in sequence). Control statements allow us to change this flow based on conditions, loops, or jumps.

ex*:* Without control statement*:*

```
console.log("Start");
console.log("Middle");
console.log("End");
```

Execution flow*:*

Start

↓

Middle

↓

End

Using control statements, we can make decisions, repeat code, or skip certain parts.

Types of Control Statements in JavaScript*:*

There are mainly three types of control statements*:*

1. Conditional Statements
2. Looping Statements
3. Jump Statements

1. Conditional Statements

Conditional statements are used to make decisions in the program.

Based on a condition (true or false), JavaScript decides which block of code should execute.

Types of Conditional Statements*:*

a) if Statement

Used when we want to execute code only when a condition is true.

Syntax*:*

```
if(condition){
   // code
}
```

Example*:*

```
let age = 20;

if(age >= 18){
   console.log("Eligible to vote");
}
```

b) if...else Statement

Used when we have two possible outcomes.

Syntax*:*

```
if(condition){
   // true block
}
else{
   // false block
}
```

Example*:*

```
let marks = 40;
```

```
if(marks >= 35){
   console.log("Pass");
}
else{
   console.log("Fail");
}
```

c) else if Statement

Used when multiple conditions need to be checked.

Example*:*

```
let score = 85;

if(score >= 90){
   console.log("Grade A");
}
else if(score >= 75){
   console.log("Grade B");
}
else if(score >= 50){
   console.log("Grade C");
}
else{
   console.log("Fail");
}
```

d) Switch Statement

Used for multiple exact value comparisons or matching.

Syntax*:*

```
switch(expression){

case value1:
   // code
   break;

case value2:
   // code
   break;

default:
   // code

}
```

Example*:*

```
let browser = "Chrome";

switch(browser){

case "Chrome":
   console.log("Chrome Browser");
   break;

case "Firefox":
   console.log("Firefox Browser");
   break;
```

default:

```
    console.log("Unknown Browser");

}
```

Output:

Chrome Browser

>> Difference Between if-else and switch

if-else	**switch**
Used for conditions and ranges	Used for exact matching
Supports logical operators	Matches specific values
Example: `age > 18`	Example: `browser == "Chrome"`

>> Note:

Use if-else when working with ranges, comparisons, and complex conditions.

Use switch when checking multiple fixed values.

Looping statements

Looping statements are used to execute a block of code repeatedly based on a condition.

1) For Loop : used when we know the iterations in advance

syntax:

```
for (initialization; condition; increment/decrement) {
//code to execute
}
```

Ex*:* To print the dropdown values

2) while Loop *:* A while loop is used when the number of iterations is unknown, and execution continues as long as the given condition is true

syntax*:*

```
while (condition) {
  // code to execute
}
```

Ex*:* Pagination

```
while(hasNextPage){
await page.locator(`.navgbutton`).click()
hasNextPage = await page.locator(`.navgbutton`).isEnabled() // Here verification
}
```

3) Do-while*:* do...while is a loop that executes the code block at least once and then checks the condition

syntax*:*

```
do {

} while (condition);
```

Ex*:* user login, ATM pin

```
do {
   // Ask the user to enter username and password

   // Validate the entered credentials

} while (credentials are invalid);
```

Used to retry the login process until valid credentials are entered, ensuring the login screen is displayed at least once before validating the credentials.

Classroom Activity 1*:*

---

1. Write a JavaScript program to print numbers that are divisible by '5' from 1 to 50
2. Print odd number from 1 to 20

strings *:* Sequence of characters

---

1. String Literal

string literal is a string value written directly in the code

ex*:*

```
let name = "John";
   let city = 'Chennai';
   let greeting = `Welcome`;
```

2. String objects

A String object is a string created using the new String() constructor.

ex*:* let str = new String("Hello");

Primitive vs Object*:*

Primitive string*:*

let a = "Welcome";

let b = "Welcome";

console.log(a === b); // O/P => true => Because primitives are compared by value.

Object string*:*

let a = new String("Welcome");

let b = new String("Welcome");

console.log(a === b); // O/P => false => Because JavaScript compares object reference (memory location).

"==="

1. reference comparison

| Type | What `===` Compares |

| --- | --- |

| Primitive | Value + Type |

| Object | Memory reference |

---

2. memory allocation

Type	Stored In Variable	Memory Location
Primitive	Actual value	Stack (generally)
Object	Reference	Heap

//String literal *:*

let companyName = "Testleaf"; // address *:* 100

let firmName = "Testleaf"; // address *:* 100

//Object literal *:*

let companyNameObj = new String(`Testleaf`) // address *:* 101

let firmNameObj = new String(`Testleaf`) // address *:* 102

Difference between concatenation "+" and template literal `${}`

Example*:*

let price = 100;

let quantity = 3;

console.log(`Total price is ${price * quantity}`);

Output

Total price is 300

Doing the same with + becomes messy*:*

console.log("Total price is " + (price * quantity));

Which is easier to read?

"User " + name + " has logged in at " + time

vs

`User ${name} has logged in at ${time}`

Week*:* Week 1

Feedback Form*:* https://forms.cloud.microsoft/r/atQhScJPrU